Лабораторная работа №4

Модули, исключения, файлы и сериализация данных в Python

1. Цель работы

Научиться делить программу на модули, обрабатывать ошибки и сохранять данные в JSON.

2. Постановка индивидуального задания

Вариант 8. Заказы. Создать систему управления заказами с номером, клиентом, товарами, стоимостью и статусом. Реализовать создание, изменение статуса, поиск, отмену, расчёт стоимости и сохранение JSON. Создать исключение OrderNotFoundError.

3. Краткие теоретические сведения

Модули разделяют модель, бизнес-логику и хранение. JSON позволяет сохранять структуры Python во внешнем файле. Контекстный менеджер with гарантирует закрытие файла. Пользовательское исключение позволяет явно сообщать об отсутствии заказа.

4. Исходный код программы

Файл exceptions.py

```
class OrderNotFoundError(Exception):
    """Заказ с указанным номером не найден."""
```

Файл main.py

```
from storage import JsonStorage
from service import OrderService
from exceptions import OrderNotFoundError

def main():
    service = OrderService(JsonStorage())
    while True:
        print("\n=== Управление заказами ===")
        print("1. Создать заказ  2. Все заказы  3. Найти заказ  4. Изменить статус  5. Отменить  0. Выход")
        choice = input("Выберите пункт: ").strip()
```

```
        try:
            if choice == "1":
                customer = input("Клиент: ").strip()
                name = input("Товар: ").strip()
                price = float(input("Цена: ")); quantity = int(input("Количество: "))
                print("Создан:", service.create(customer, [{"name": name, "price": price, "quantity": quantity}]))
            elif choice == "2":
                for order in service.all_orders(): print(order)
            elif choice == "3": print(service.search(int(input("Номер заказа: "))))
            elif choice == "4": print(service.change_status(int(input("Номер заказа: ")), input("Новый статус: ").strip()))
            elif choice == "5": print(service.cancel(int(input("Номер заказа: "))))
            elif choice == "0": break
            else: print("Неизвестный пункт меню.")
        except (ValueError, OrderNotFoundError) as exc: print(f"Ошибка: {exc}")

if __name__ == "__main__": main()
```

Файл models.py

```
from dataclasses import dataclass, asdict

@dataclass
class Order:
    number: int
    customer: str
    items: list
    status: str = "новый"

    @property
    def total(self):
        return round(sum(float(item["price"]) * int(item["quantity"]) for item in self.items), 2)

    def to_dict(self):
        data = asdict(self); data["total"] = self.total; return data
```

Файл service.py

```
from models import Order
from exceptions import OrderNotFoundError

class OrderService:
    def __init__(self, storage):
        self.storage = storage
        self.orders = storage.load()

    def create(self, customer, items):
        if not customer.strip(): raise ValueError("Имя клиента не должно быть пустым.")
        if not items: raise ValueError("Заказ должен содержать хотя бы один товар.")
        for item in items:
            if float(item["price"]) < 0 or int(item["quantity"]) <= 0:
                raise ValueError("Цена должна быть неотрицательной, количество — положительным.")
        number = max((int(order["number"]) for order in self.orders), default=0) + 1
        order = Order(number, customer.strip(), items)
        self.orders.append(order.to_dict()); self.storage.save(self.orders)
        return order.to_dict()

    def _find(self, number):
        for order in self.orders:
            if int(order["number"]) == int(number): return order
        raise OrderNotFoundError(f"Заказ №{number} не найден.")
```

```
    def change_status(self, number, status):
        allowed = {"новый", "оплачен", "отправлен", "отменён"}
        if status not in allowed: raise ValueError("Недопустимый статус заказа.")
        order = self._find(number); order["status"] = status; self.storage.save(self.orders); return order

    def cancel(self, number): return self.change_status(number, "отменён")
    def search(self, number): return self._find(number)
    def all_orders(self): return list(self.orders)
```

Файл storage.py

```
import json
from pathlib import Path

class JsonStorage:
    def __init__(self, path="orders.json"):
        self.path = Path(path)

    def load(self):
        if not self.path.exists(): return []
        try:
            with self.path.open("r", encoding="utf-8") as file:
                data = json.load(file)
            if not isinstance(data, list): raise ValueError("Корневое значение JSON должно быть списком.")
            return data
        except json.JSONDecodeError as exc:
            raise ValueError(f"Файл JSON повреждён: {exc}") from exc

    def save(self, orders):
        with self.path.open("w", encoding="utf-8") as file:
            json.dump(orders, file, ensure_ascii=False, indent=2)
```

5. Объяснение работы программы

Order хранит данные заказа и рассчитывает стоимость позиций. JsonStorage загружает и сохраняет JSON. OrderService выполняет бизнес-операции, проверяет входные данные и использует OrderNotFoundError. main.py отвечает за меню и перехватывает ожидаемые ошибки.

6. Примеры работы программы в виде снимков экрана

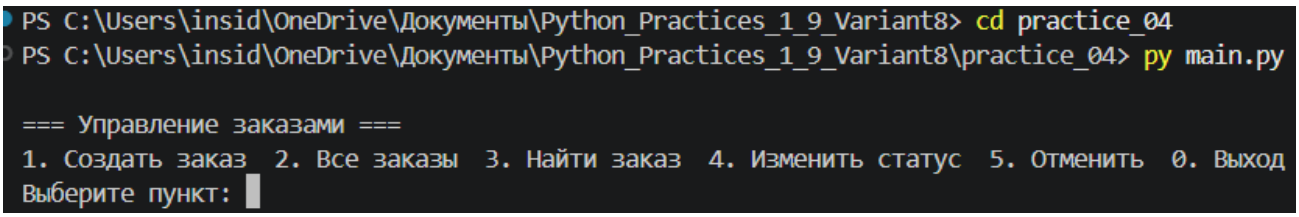


Рисунок 4.1 – Главное меню управления заказами.

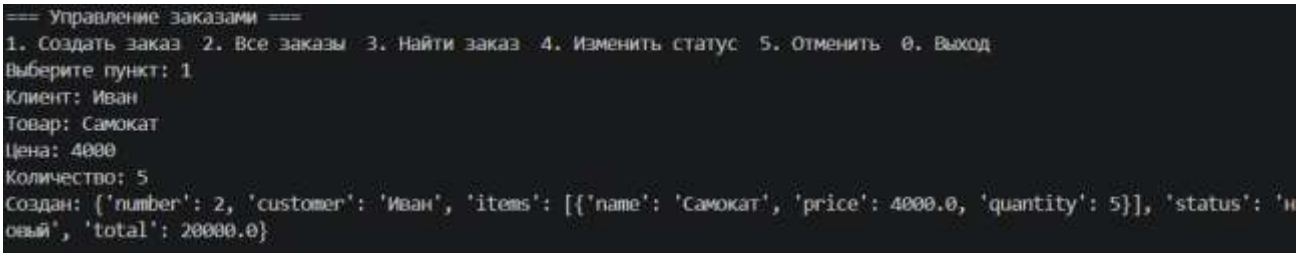


Рисунок 4.2 – Создание заказа и сохранение данных в orders.json.

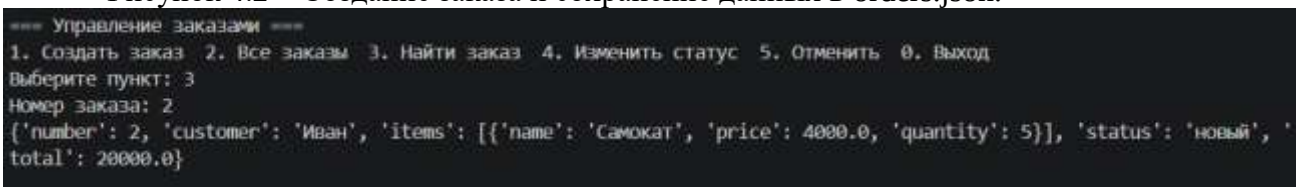


Рисунок 4.3 – Поиск заказа, изменение статуса или сообщение об ошибке при отсутствии номера.

7. Вывод

В ходе выполнения лабораторной работы были реализованы требования индивидуального задания варианта 8. Order хранит данные заказа и рассчитывает стоимость позиций. JsonStorage загружает и сохраняет JSON. OrderService выполняет бизнес-операции, проверяет входные данные и использует OrderNotFoundError. main.py отвечает за меню и перехватывает ожидаемые ошибки. Программа разделена на логические компоненты, содержит проверку типичных ошибок и сопровождается автоматическими тестами.

Выполнил: Куршаков Егор Игоревич